QR Code and Barcode Detection System

ILEGH SALMA AND ELHAIRECH AMINE

1. Introduction

The objective of this project is to design and implement a system capable of detecting and decoding QR codes and barcodes from images using computer vision techniques. The system is developed using Python and OpenCV, and supports both automated processing through a pipeline and interactive usage via a graphical interface. The main challenge lies in accurately detecting codes under different conditions such as noise, lighting variations, and partial occlusion.

2. Methodology (Pipeline)

The system is structured as a multi-stage pipeline, where each step contributes to improving detection accuracy and robustness.

2.2 Image Loading

Images are loaded using OpenCV (cv2.imread) or via an interactive interface. Each image is processed as a NumPy array in BGR format.

2.2 Preprocessing

Preprocessing is a critical stage that prepares the image for detection by enhancing relevant features and reducing noise. The following operations are applied:

-**Grayscale conversion**; Reduces computational complexity by removing color information while preserving intensity variations.
-**Gaussian Blur**: Smooths the image to reduce high-frequency noise and improve edge detection stability.
-**Otsu Thresholding:** Automatically determines an optimal threshold to convert the image into a binary representation, which is particularly useful for QR code detection.
-**Canny Edge Detection**: Extracts edges that are later used for contour-based detection of QR codes.
-**Sobel Gradient (Barcode-specific):** Computes directional gradients to emphasize vertical edges, which correspond to barcode structures.
-**Morphological Operations:** Applied to refine shapes:

- For QR: clean and connect edges.
- For barcode: group parallel lines into a single region

This stage produces multiple representations of the image (grayscale, thresholded, edges, gradient, closed), each used in later steps.

2.3 QR Code Detection

QR code detection is performed using a two-level strategy:

- **Global Detection**

-Uses OpenCV's QRCodeDetector on the full image
-Tests multiple versions (original, grayscale, thresholded)
-Fast and effective for clear and large QR codes

- **Local Detection**

-Activated when global detection fails
-Uses contour detection to identify candidate regions
-Filtres contours based on:

- Area
- Shape (square-like)

-Extracts ROIs and attempts decoding

This dual approach ensures both efficiency and robustness.

2.4 Barcode Detection

Barcode detection follows a different approach due to its structural nature:

-Sobel Gradient highlights vertical edges
-Thresholding converts gradient into binary form
-Morphological Closing connects barcode lines into a continuous region
-Contour Detection identifies candidate barcode regions

Only significant contours (based on area) are considered to avoid noise.

2.5 ROI Extraction

For each detected region (QR or barcode), a Region of Interest (ROI) is extracted using bounding boxes.
This step reduces the search space and improves decoding accuracy by focusing only on relevant areas.

2.6 Decoding

Decoding is performed using different methods:

- QR Codes:
  Decoded using OpenCV's QRCodeDetector
- Barcodes:
  Decoded using the pyzbar library

To improve robustness, multiple versions of each ROI are tested:

- original
- grayscale
- histogram-equalized
- thresholded
- resized

This multi-representation strategy significantly increases decoding success rates.

2.7 Visualization

The final step consists of visualizing the results:
-QR codes are highlighted using polygons
-Barcodes are highlighted using bounding rectangles
-Decoded data is displayed alongside the image
-This allows easy verification of detection accuracy.

3. OpenCV Functions and Concepts

The implementation relies on several fundamental computer vision concepts:

Canny Edge Detection: Detects edges using gradient magnitude and thresholding.
Sobel Operator: Computes directional gradients (used for barcode detection)
Otsu Thresholding: Automatically selects optimal threshold values.
Morphological Operations: Refine shapes and connect fragmented structures.
Contour Detection (find Contours) :Extracts candidate object boundaries.
Bounding Boxes: Used for ROI extraction and localization.

These functions correspond to classical algorithms studied in computer vision courses.

4. Challenges

Several practical challenges were encountered:

-Noise and blur affecting edge detection
-Reflections on surfaces (e.g., plastic bottles)
-Broken or distorted barcode lines
-Sensitivity to preprocessing parameters (kernel size, thresholds)
-Large or imprecise ROI selection, reducing decoding accuracy
-Complex backgrounds interfering with QR detection

Addressing these challenges required careful tuning of preprocessing and detection strategies.

5. Results and Evaluation

The system was evaluated on multiple test images containing both QR codes and barcodes.

Figure 1:This figure shows the successful detection and decoding of an EAN-13 barcode. The system correctly identifies the barcode region using gradient-based edge detection (Sobel operator) and morphological processing. The bounding box accurately localizes the the barcode, and the decoding is performed using the pyzbar library.

Figure 2:This figure presents the detection of a CODE128 barcode. The system successfully extracts the region of interest (ROI) and decodes the embedded information. The use of thresholding and morphological closing helps in grouping the barcode lines into a single region

```
Image loaded: /content/QR1.png | Shape: (225, 225, 3)
```

Detection Results

```
=== DETECTED CODES ===
• [QR - Global] https://onkruitvergaatniet.nl/qr-pierson
```

Figure 3: This figure shows a standard QR code detected using OpenCV's QRCodeDetector. The global detection method successfully identifies and decodes the QR code without requiring additional processing.

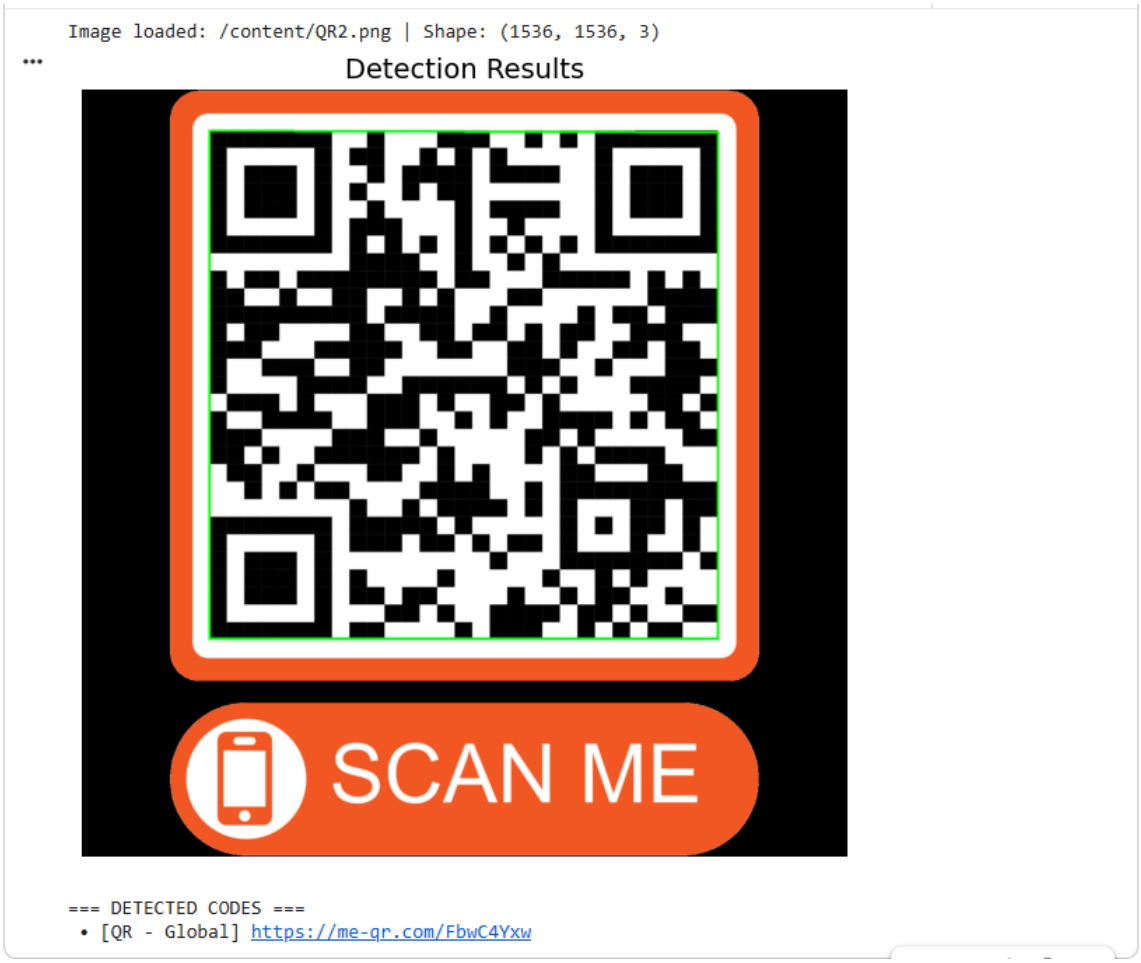

Figure 4: This figure illustrates the detection of a QR code with additional graphical elements (logo and "SCAN ME" label). Despite the added complexity, the system successfully detects and decodes the QR code.

6. Conclusion

In conclusion, the project successfully demonstrates how classical computer vision techniques can be used to detect and decode QR codes and barcodes.

The combination of preprocessing, edge detection, morphological operations, and contour analysis provides a robust solution. Future improvements could include handling rotated images, improving detection in low-light conditions, and integrating deep learning techniques.